

AG35-CET&AG35-EUT

QuecOpen(SDK) eCall 开发指导

LTE Standard 模块系列

版本：1.0.1

日期：2025-03-06

状态：临时文件



上海移远通信技术股份有限公司（以下简称“移远通信”）始终以为客户提供最及时、最全面的服务为宗旨。如需任何帮助，请随时联系我司上海总部，联系方式如下：

上海移远通信技术股份有限公司

上海市闵行区田林路 1016 号科技绿洲 3 期（B 区）5 号楼 邮编：200233

电话：+86 21 5108 6236 邮箱：info@quectel.com

或联系我司当地办事处，详情请登录：<http://www.quectel.com/cn/support/sales.htm>。

如需技术支持或反馈我司技术文档中的问题，请随时登录网址：

<http://www.quectel.com/cn/support/technical.htm> 或发送邮件至：support@quectel.com。

前言

移远通信提供该文档内容以支持客户的产品设计。客户须按照文档中提供的规范、参数来设计产品。同时，您理解并同意，移远通信提供的参考设计仅作为示例。您同意在设计您目标产品时使用您独立的分析、评估和判断。在使用本文档所指导的任何硬软件或服务之前，请仔细阅读本声明。您在此承认并同意，尽管移远通信采取了商业范围内的合理努力来提供尽可能好的体验，但本文档和其所涉及服务是在“可用”基础上提供给您的。移远通信可在未事先通知的情况下，自行决定随时增加、修改或重述本文档。

使用和披露限制

许可协议

除非移远通信特别授权，否则我司所提供硬软件、材料和文档的接收方须对接收的内容保密，不得将其用于除本项目的实施与开展以外的任何其他目的。

版权声明

移远通信产品和本协议项下的第三方产品可能包含受移远通信或第三方材料、硬软件和文档版权保护的相关资料。除非事先得到书面同意，否则您不得获取、使用、向第三方披露我司所提供的文档和信息，或对此类受版权保护的资料进行复制、转载、抄袭、出版、展示、翻译、分发、合并、修改，或创造其衍生作品。移远通信或第三方对受版权保护的资料拥有专有权，不授予或转让任何专利、版权、商标或服务商标权的许可。为避免歧义，除了正常的非独家、免版税的产品使用许可，任何形式的购买都不可被视为授予许可。对于任何违反保密义务、未经授权使用或以其他非法形式恶意使用所述文档和信息的违法侵权行为，移远通信有权追究法律责任。

商标

除另行规定，本文档中的任何内容均不授予在广告、宣传或其他方面使用移远通信或第三方的任何商标、商号及名称，或其缩略语，或其仿冒品的权利。

第三方权利

您理解本文档可能涉及一个或多个属于第三方的硬软件和文档（“第三方材料”）。您对此类第三方材料的使用应受本文档的所有限制和义务约束。

移远通信针对第三方材料不做任何明示或暗示的保证或陈述，包括但不限于任何暗示或法定的适销性或特定用途的适用性、平静受益权、系统集成、信息准确性以及与许可技术或被许可人使用许可技术相关的不侵犯任何第三方知识产权的保证。本协议中的任何内容都不构成移远通信对任何移远通信产品或任何其他软硬件、设备、工具、信息或产品的开发、增强、修改、分销、营销、销售、提供销售或以其他方式维持生产的陈述或保证。此外，移远通信免除因交易过程、使用或贸易而产生的任何和所有保证。

隐私声明

为实现移远通信产品功能，特定设备数据将会上传至移远通信或第三方服务器（包括运营商、芯片供应商或您指定的服务器）。移远通信严格遵守相关法律法规，仅为实现产品功能之目的或在适用法律允许的情况下保留、使用、披露或以其他方式处理相关数据。当您与第三方进行数据交互前，请自行了解其隐私保护和数据安全政策。

免责声明

- 1) 移远通信不承担任何因未能遵守有关操作或设计规范而造成损害的责任。
- 2) 移远通信不承担因本文档中的任何因不准确、遗漏、或使用本文档中的信息而产生的任何责任。
- 3) 移远通信尽力确保开发中功能的完整性、准确性、及时性，但不排除上述功能错误或遗漏的可能。除非另有协议规定，否则移远通信对开发中功能的使用不做任何暗示或法定的保证。在适用法律允许的最大范围内，移远通信不对任何因使用开发中功能而遭受的损害承担责任，无论此类损害是否可以预见。
- 4) 移远通信对第三方网站及第三方资源的信息、内容、广告、商业报价、产品、服务和材料的可访问性、安全性、准确性、可用性、合法性和完整性不承担任何法律责任。

版权所有 ©上海移远通信技术股份有限公司 2025，保留一切权利。

Copyright © Quectel Wireless Solutions Co., Ltd. 2025.

文档历史

本文档为临时版本，其中涉及的模块引脚或接口定义、频段、功能、特性及设计等（若有）发生变动的可能性较大，部分参数经过初步验证或仍未验证。变更或补充信息将在后续正式版本中体现。临时版本仅供临时参考使用，若与正式版本存在差异，应以最新版本为准。修订记录如下所示：

修订记录

版本	日期	作者	变更表述
-	2023-09-27	Lyndsay XIE	文档创建
1.0.0	2023-09-27	Lyndsay XIE	临时版本
			临时文档
			1. 新增适用模块 AG35-EUT。
			2. 更新 eCall 重拨机制欧洲标准（第 2.4 章）。
			3. 更新 eCall 状态事件列表（第 2.6 章）。
			4. 移除结构体 msd_t（第 3.3.4 章）。
1.0.1	2025-03-06	Lyndsay XIE	5. 更新结构体 ql_ecall_config_t（第 3.3.11.1 章）。
			6. 更新结构体 msd_Vin_t（第 3.3.16.2 章）。
			7. 更新车辆类型（第 3.3.25、3.3.26 章）。
			8. 新增函数返回值（第 3.3.27~3.3.31 章）。
			9. 更新推荐的函数使用流程（第 3.4.1 章）。
			10. 更新设置回调函数上报 eCall 状态的示例（第 3.4.2.2 章）。

目录

文档历史	3
目录	4
表格索引	6
1 引言	7
2 eCall 综述.....	8
2.1. eCall 工作流程.....	8
2.1.1. 流程概述	8
2.1.2. 流程示意图	9
2.2. eCall 触发模式.....	9
2.3. MO eCall 与 MT eCall	9
2.4. eCall 重拨机制.....	9
2.5. eCall 标准定时器	10
2.6. eCall 状态事件.....	10
3 eCall API.....	12
3.1. 头文件.....	12
3.2. 函数概览.....	12
3.3. 函数详解.....	14
3.3.1. ql_ecall_init.....	14
3.3.2. ql_ecall_deinit	14
3.3.3. ql_ecall_set_user_ind_cb	15
3.3.3.1. ql_ecall_user_ind_f cb	15
3.3.4. ql_ecall_dial.....	16
3.3.4.1. ql_voice_ecall_info_t.....	16
3.3.4.2. QL_VOICE_ECALL_TYPE_E	17
3.3.4.3. ECALL_SYSTEM_STD_E	17
3.3.5. ql_ecall_update_msd_raw	18
3.3.6. ql_ecall_update_msd	18
3.3.7. ql_ecall_hangup	19
3.3.8. ql_ecall_start_test	19
3.3.9. ql_ecall_start_manual	20
3.3.10. ql_ecall_start_automatic	20
3.3.11. ql_ecall_set_config_info.....	21
3.3.11.1. ql_ecall_config_t	21
3.3.12. ql_ecall_get_config_info	23
3.3.13. ql_ecall_set_test_number	23
3.3.14. ql_ecall_set_system_std.....	24
3.3.15. ql_ecall_get_system_std.....	24
3.3.16. ql_ecall_set_msd_vin.....	25
3.3.16.1. QL_ERR_MSD_RESULT_E	25
3.3.16.2. msd_Vin_t	27

3.3.17.	ql_ecall_get_msd_vin.....	27
3.3.18.	ql_ecall_set_msd_version.....	28
3.3.19.	ql_ecall_get_msd_version	28
3.3.20.	ql_ecall_set_msd_tx_mode	28
3.3.20.1.	QL_ECALL_MSD_TX_MODE_E	29
3.3.21.	ql_ecall_get_msd_tx_mode	29
3.3.22.	ql_ecall_set_msd_position.....	30
3.3.23.	ql_ecall_set_msd_position_n1.....	30
3.3.24.	ql_ecall_set_msd_position_n2.....	31
3.3.25.	ql_ecall_set_msd_vehicle_type	31
3.3.26.	ql_ecall_get_msd_vehicle_type.....	32
3.3.27.	ql_ecall_set_msd_passengers_count.....	33
3.3.28.	ql_ecall_set_msd_propulsion_type.....	34
3.3.29.	ql_ecall_get_msd_propulsion_type	34
3.3.30.	ql_ecall_set_msd_call_type	35
3.3.31.	ql_ecall_get_msd_call_type.....	35
3.4.	函数使用	36
3.4.1.	推荐的函数使用流程.....	36
3.4.2.	函数使用示例.....	37
3.4.2.1.	初始化 eCall	37
3.4.2.2.	设置回调函数上报 eCall 状态	37
3.4.2.3.	拨打 eCall	39
3.4.2.4.	使用原始数据更新 MSD	40
3.4.2.5.	更新 MSD	41
3.4.2.6.	挂断 eCall	42
3.4.2.7.	设置车辆识别码	42
3.4.2.8.	发起 eCall 测试	43
3.4.2.9.	发起手动拨打 eCall.....	43
3.4.2.10.	发起自动拨打 eCall.....	44
4	附录 参考文档及术语缩写	45

表格索引

表 1: eCall 标准定时器	10
表 2: eCall 状态事件列表.....	10
表 3: 函数概览.....	12
表 4: 参考文档.....	45
表 5: 术语缩写	45

Preliminary
Quectel Confidential

1 引言

移远通信 AG35-CET 和 AG35-EUT 模块支持 QuecOpen® 方案；QuecOpen® 是基于 Linux 的嵌入式开发平台，可简化 IoV 应用的软件设计和开发过程。有关 QuecOpen® 方案的详细信息，请参考文档[1]。

本文档适用于 SDK 构建环境的 QuecOpen® 方案。AG35-CET 和 AG35-EUT 模块在 GSM 或 UMTS 网络下可实现 eCall 功能。本文档主要介绍 eCall 的工作流程、配置参数等，以及如何通过 API 函数应用和实现 eCall 功能。

Preliminary
Quectel Confidential

2 eCall 综述

2.1. eCall 工作流程

eCall 是一项欧洲倡议，旨在为欧盟内任何地方发生碰撞的驾车者提供快速援助。eCall 倡议计划在所有车辆中部署安装一台设备，该设备在发生严重交通事故时将自动拨打国际求救电话（112），并通过无线通信发送乘客与车辆信息，以及 GNSS 定位系统坐标到当地应急机构；其也支持手动拨打求救电话。

2.1.1. 流程概述

当发生交通事故时，车辆的 eCall 系统将通过卫星定位装置获取该车辆的位置信息，自动/手动建立紧急语音呼叫，并通过 IVS（包含移远通信 AG35-CET 或 AG35-EUT 模块的车载设备）将包含位置、乘客数量、车辆识别码等的 MSD（最小数据集）消息集合通过移动蜂窝网络发送至当地公共安全应急机构，如 PSAP，从而在短时间内完成对事故车辆相关信息的采集，以便开展紧急救援行动。

由此可见，eCall 系统通常包含车载系统（IVS）和公共安全应答中心（PSAP）。

- IVS 主要用于发起或者接听 eCall 以及发送 MSD 等；发生车祸时，IVS 自动拨打紧急呼救电话 112，同时通过语音通道将 MSD 发送到 PSAP。MSD 包含的内容可以有：位置信息、时间、乘客数量、车牌号，以及其他紧急救援所需要的信息。
- PSAP 主要负责接听或回拨 eCall 电话，请求 MSD 数据以及发起公共救援等。

备注

1. 本文档将移远通信 AG35-CET 或 AG35-EUT QuecOpen 模块作为 IVS 使用。
2. 由于中国目前还未建立标准的 PSAP，所以无法在实网下进行测试；但可以使用实验室内的标准测试仪器 CMW500 及其配套软件进行模拟。本文档使用标准测试仪器 CMW500 作为模拟 PSAP 进行测试。

2.1.2. 流程示意图

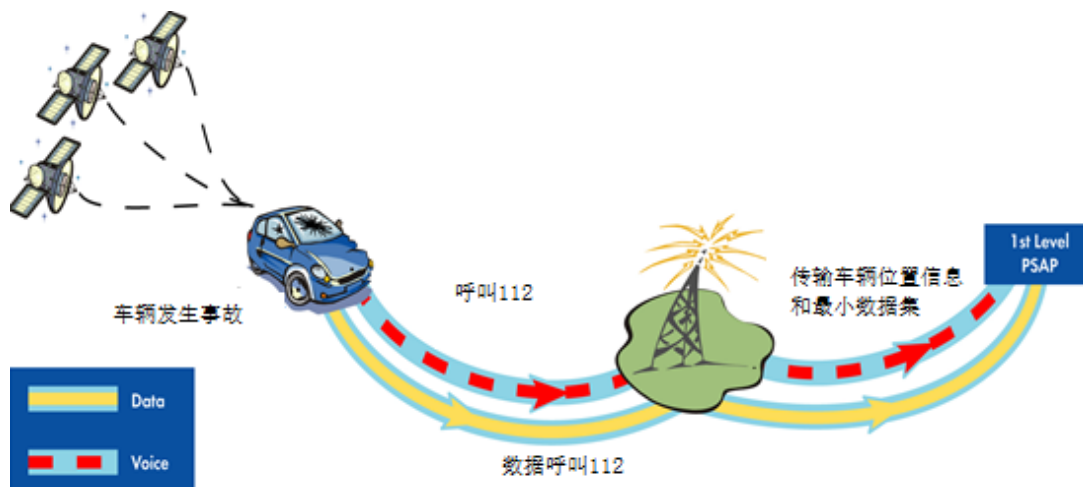


图 1: eCall 流程示意图

2.2. eCall 触发模式

eCall 支持手动和自动触发两种方式，可通过 `ql_ecall_dial()` 函数进行配置（详见 `ql_voice_ecall_info_t` 的 `auto_trigger` 参数）。

2.3. MO eCall 与 MT eCall

MO eCall 是指 IVS 主动发起 eCall 通话；当 PSAP 接听后，IVS 会通过 PUSH 模式自动发送 MSD，整个流程无需 PSAP 发送请求。

MT eCall 是指前一次的 eCall 通话由 PSAP 主动挂断，此时只要 T2 定时器还没到期，IVS 就会自动接听 PSAP 主动发起的 eCall 通话。且本次 eCall 使用 PULL 模式进行 MSD 传输，即需要 PSAP 主动发送 MSD 请求。

2.4. eCall 重拨机制

eCall 拨打失败或 eCall 通话异常断开后，IVS 会按如下机制自动重拨：

- 欧洲标准：间隔 60 秒重拨一次，共重拨 10 次；通话中异常断连会立即重拨，共 2 次；
- GLONASS 标准：间隔 30 秒重拨一次，共重拨 10 次。

备注

用户可按需修改重拨次数和时间。

2.5. eCall 标准定时器

eCall 标准定时器的设置遵循《BS EN 16454-2015》的要求，详情如下所示：

表 1：eCall 标准定时器

定时器	定时器功能说明	默认值
T2	IVS 的 eCall 通话超时时间（eCall 通话持续时间超过此定时器时间时，IVS 将主动挂断通话）	3600 秒
T3	IVS 初始化信号持续时间	2 秒
T5	IVS 等待发送 MSD 的时间	5 秒
T6	IVS 等待接收 AL-ACK 的时间	5 秒
T7	IVS 传输 MSD 的最大时间	20 秒
T9	IVS 在模式下最小的驻网持续时间	3600 秒
T10	IVS 在 eCall only 模式下最大的驻网持续时间	12 小时

2.6. eCall 状态事件

eCall 状态事件是指从 eCall 的发起到结束期间，模块上报到 AP 的 eCall 状态信息。具体事件定义在头文件 `ql-sysroots/usr/include/ql-sdk/ql_ecall.h` 中。

表 2：eCall 状态事件列表

eCall 状态事件	说明
QL_ECALL_EVENT_T2_TIMEOUT	T2 超时
QL_ECALL_EVENT_T5_TIMEOUT	T5 超时

QL_ECALL_EVENT_T6_TIMEOUT	T6 超时
QL_ECALL_EVENT_T7_TIMEOUT	T7 超时
QL_ECALL_EVENT_SENDING_START	IVS 开始发送 MSD
QL_ECALL_EVENT_SENDING_MSD	IVS 正在发送 MSD
QL_ECALL_EVENT_LLACK_RECEIVED	IVS 收到 LL-ACK 消息
QL_ECALL_EVENT_ALLACK_POSITIVE_RECEIVED	IVS 收到 AL-ACK 消息
QL_ECALL_EVENT_ALLACK_CLEARDOWN_RECEIVED	IVS 收到 PSAP CLEARDOWN 消息
QL_ECALL_EVENT_MSD_TRANS_STATUS_SUCCESS	MSD 发送成功
QL_ECALL_EVENT_MSD_TRANS_STATUS_FAILURE	MSD 发送失败
QL_ECALL_EVENT_ECALL_STARTED	发起 eCall
QL_ECALL_EVENT_MSDUPDATE_PSAP_REQUIRE_UPDATE_MSD	PSAP 请求更新 MSD
QL_ECALL_EVENT_MSDUPDATE_PSAP_REQUIRE_TIMEOUT	MSD 更新超时
QL_ECALL_EVENT_ACTIVE	eCall 通话连接成功
QL_ECALL_EVENT_DISCONNECTED	eCall 通话正常断开连接
QL_ECALL_EVENT_ABNORMAL_HANGUP	IVS 异常挂断
QL_ECALL_EVENT_PSAP_CALLBACK_START	开启 PSAP 回拨自动接听
QL_ECALL_EVENT_INCOMING_CALL	IVS 接收到 PSAP 回拨
QL_ECALL_EVENT_DIAL_DURATION_TIMEOUT	eCall 重拨超时
QL_ECALL_EVENT_INTERVAL_TIMEOUT	IVS 重拨间隔时间超时
QL_ECALL_EVENT_AUTO_ANSWER_TIMEOUT	自动接听超时

3 eCall API

3.1. 头文件

eCall API 头文件为 `ql_ecall.h`，位于 SDK 包的 `ql-sysroots/usr/include/ql-sdk` 路径下。若无特别说明，本文档所涉及头文件均在该目录下。

3.2. 函数概览

表 3：函数概览

函数	说明
<code>ql_ecall_init()</code>	初始化 eCall
<code>ql_ecall_deinit()</code>	注销 eCall
<code>ql_ecall_set_user_ind_cb()</code>	注册 eCall 状态上报回调函数
<code>ql_ecall_dial()</code>	发起 eCall 通话
<code>ql_ecall_update_msd_raw()</code>	使用原始数据更新 MSD
<code>ql_ecall_update_msd()</code>	更新 MSD
<code>ql_ecall_hangup()</code>	挂断 eCall 通话
<code>ql_ecall_start_test()</code>	发起 eCall 测试
<code>ql_ecall_start_manual()</code>	发起手动拨打 eCall
<code>ql_ecall_start_automatic()</code>	发起自动拨打 eCall
<code>ql_ecall_set_config_info()</code>	配置 eCall
<code>ql_ecall_get_config_info()</code>	获取 eCall 配置
<code>ql_ecall_set_test_number()</code>	设置 eCall 测试号码

<code>ql_ecall_set_system_std()</code>	设置 eCall 系统标准类型
<code>ql_ecall_get_system_std()</code>	获取 eCall 系统标准类型
<code>ql_ecall_set_msd_vin()</code>	设置车辆识别码
<code>ql_ecall_get_msd_vin()</code>	获取车辆识别码
<code>ql_ecall_set_msd_version()</code>	设置 MSD 版本号
<code>ql_ecall_get_msd_version()</code>	获取 MSD 版本号
<code>ql_ecall_set_msd_tx_mode()</code>	设置 MSD 发送模式
<code>ql_ecall_get_msd_tx_mode()</code>	获取 MSD 发送模式
<code>ql_ecall_set_msd_position()</code>	设置车辆位置信息
<code>ql_ecall_set_msd_position_n1()</code>	设置车辆经纬度的增量 N1
<code>ql_ecall_set_msd_position2_n2()</code>	设置车辆经纬度的增量 N2
<code>ql_ecall_set_msd_vehicle_type()</code>	设置车辆类型
<code>ql_ecall_get_msd_vehicle_type()</code>	获取车辆类型
<code>ql_ecall_set_msd_passengers_count()</code>	设置乘客数量
<code>ql_ecall_set_msd_propulsion_type()</code>	设置车辆推动类型
<code>ql_ecall_get_msd_propulsion_type()</code>	获取车辆推动类型
<code>ql_ecall_set_msd_call_type()</code>	设置 eCall 类型
<code>ql_ecall_get_msd_call_type()</code>	获取 eCall 类型

备注

若无特别说明，所有 eCall API 均不支持并发调用，并且不能在相关回调函数中调用以上 API。

3.3. 函数详解

3.3.1. ql_ecall_init

该函数用于初始化 eCall。

- 函数原型

```
int ql_ecall_init(void);
```

- 参数

无

- 返回值

QL_ERR_OK	函数执行成功
其他值	函数执行失败，错误码详见 <i>ql_type.h</i> 。

备注

使用其他 eCall API 前，必须调用该函数以初始化 eCall 服务。

3.3.2. ql_ecall_deinit

该函数用于注销 eCall。

- 函数原型

```
int ql_ecall_deinit(void);
```

- 参数

无

- 返回值

QL_ERR_OK	函数执行成功
其他值	函数执行失败，错误码详见 <i>ql_type.h</i> 。

3.3.3. ql_ecall_set_user_ind_cb

该函数用于注册 eCall 状态上报回调函数。

- 函数原型

```
void ql_ecall_set_user_ind_cb(ql_ecall_user_ind_f cb, void *userdata);
```

- 参数

cb:

[In] eCall 状态回调函数；详见第 3.3.3.1 章。

userdata:

[In] 用户数据。通常为 *NULL*。

- 返回值

无

3.3.3.1. ql_ecall_user_ind_f cb

该回调函数用于处理 eCall 状态变化。

- 函数原型

```
typedef void (*ql_ecall_user_ind_f)(int ind, void *userdata);
```

- 参数

ind:

[In] eCall 状态；详见表 2。

userdata:

[In] 用户数据。

- 返回值

无

3.3.4. ql_ecall_dial

该函数用于发起 eCall 通话。

● 函数原型

```
int ql_ecall_dial(int sim_id, const ql_voice_ecall_info_t* const p_info, uint32_t* const p_id);
```

● 参数

sim_id:

[In] (U)SIM 卡 ID。

- 0 无效卡槽
- 1 (U)SIM1
- 2 (U)SIM2

p_info:

[In] eCall 输入信息；详见第 3.3.4.1 章。暂不支持 *msd* 参数输入。

p_id:

[Out] 返回的 eCall ID。

● 返回值

QL_ERR_OK 函数执行成功

其他值 函数执行失败，错误码详见 *ql_type.h*。

3.3.4.1. ql_voice_ecall_info_t

eCall 输入信息结构体定义如下：

```
typedef struct
{
    uint32_t          msd_len;
    uint8_t          msd[QL_VOICE_MAX_ECALL_MSD];
    QL_VOICE_ECALL_TYPE_E type;
    int              auto_trigger;
    char              test_number[QL_VOICE_MAX_PHONE_NUMBER];
} ql_voice_ecall_info_t;
```

● 参数

类型	参数	描述
uint32_t	<i>msd_len</i>	MSD 数据长度

uint8_t	msd[QL_VOICE_MAX_ECALL_MSDD]	MSD 数据
QL_VOICE_ECALL_TYPE_E	type	eCall 类型；详见第 3.3.4.2 章。
int	auto_trigger	eCall 触发方式 0 手动 1 自动
char	test_number[QL_VOICE_MAX_PHONE_NUMBER]	测试号码

3.3.4.2. QL_VOICE_ECALL_TYPE_E

eCall 类型枚举定义如下：

```
typedef enum
{
    QL_VOICE_ECALL_TYPE_TEST           = 1,
    QL_VOICE_ECALL_TYPE_EMERGENCY      = 2,
    QL_VOICE_ECALL_TYPE_RECONFIG       = 3,
} QL_VOICE_ECALL_TYPE_E;
```

● 成员

成员	描述
QL_VOICE_ECALL_TYPE_TEST	测试模式（可指定 PSAP 号码进行拨打）
QL_VOICE_ECALL_TYPE_EMERGENCY	正常模式（默认拨打紧急救助电话 112）
QL_VOICE_ECALL_TYPE_RECONFIG	重新配置模式

3.3.4.3. ECALL_SYSTEM_STD_E

eCall 系统标准类型枚举定义如下：

```
typedef enum
{
    ECALL_SYSTEM_STD_PAN_EUROPEAN,
    ECALL_SYSTEM_STD_EU_GLOASS
} ECALL_SYSTEM_STD_E;
```

● 成员

成员	描述
ECALL_SYSTEM_STD_PAN_EUROPEAN	欧洲标准

ECALL_SYSTEM_STD_ERA_GLONASS

GLONASS 标准（暂不支持）

3.3.5. ql_ecall_update_msd_raw

该函数用于使用原始数据更新 MSD。

- 函数原型

```
int ql_ecall_update_msd_raw(uint8_t* msd, uint32_t msd_len);
```

- 参数

msd:

[In] 待更新的 MSD。

msd_len:

[In] MSD 长度。单位：hex，最大长度：140 hex。

- 返回值

QL_ERR_OK 函数执行成功

其他值 函数执行失败，错误码详见 *ql_type.h*。

3.3.6. ql_ecall_update_msd

该函数用于更新 MSD，需要配套其他 MSD 参数设置接口使用。

- 函数原型

```
int ql_ecall_update_msd(void);
```

- 参数

无

- 返回值

QL_ERR_OK 函数执行成功

其他值 函数执行失败，错误码详见 *ql_type.h*。

3.3.7. ql_ecall_hangup

该函数用于挂断 eCall 通话。

- 函数原型

```
int ql_ecall_hangup(void);
```

- 参数

无

- 返回值

QL_ERR_OK 函数执行成功
其他值 函数执行失败，错误码详见 *ql_type.h*。

3.3.8. ql_ecall_start_test

该函数用于发起 eCall 测试。

- 函数原型

```
int ql_ecall_start_test(int sim_id);
```

- 参数

sim_id:

[In] (U)SIM 卡 ID。

- 0 无效卡槽
- 1 (U)SIM1
- 2 (U)SIM2

- 返回值

QL_ERR_OK 函数执行成功
其他值 函数执行失败，错误码详见 *ql_type.h*。

备注

调用该函数之前，需调用函数完成全部 MSD 信息配置，其中车辆识别码必须设置。

3.3.9. ql_ecall_start_manual

该函数用于发起手动拨打 eCall。

- 函数原型

```
int ql_ecall_start_manual(int sim_id);
```

- 参数

sim_id:

[In] (U)SIM 卡 ID。

- 0 无效卡槽
- 1 (U)SIM1
- 2 (U)SIM2

- 返回值

QL_ERR_OK 函数执行成功

其他值 函数执行失败，错误码详见 *ql_type.h*。

备注

调用该函数之前，需调用函数完成全部 MSD 信息配置，其中车辆识别码必须设置。

3.3.10. ql_ecall_start_automatic

该函数用于发起自动拨打 eCall。

- 函数原型

```
int ql_ecall_start_automatic(int sim_id);
```

- 参数

sim_id:

[In] (U)SIM 卡 ID。

- 0 无效卡槽
- 1 (U)SIM1
- 2 (U)SIM2

- 返回值

`QL_ERR_OK` 函数执行成功
其他值 函数执行失败，错误码详见 `ql_type.h`。

备注

调用该函数之前，需调用函数完成全部 MSD 信息配置，其中车辆识别码必须设置。

3.3.11. ql_ecall_set_config_info

该函数用于配置 eCall。

- 函数原型

```
void ql_ecall_set_config_info(ql_ecall_config_t ecall_context_info);
```

- 参数

ecall_context_info:

[In] eCall 配置信息；详见第 3.3.11.1 章。

- 返回值

无

3.3.11.1. ql_ecall_config_t

eCall 配置信息结构体定义如下：

```
typedef struct
{
    uint8_t      ecallonly_valid;
    uint8_t      ecallonly;
    uint8_t      cleardownTimer_timeout_ms_valid;
    uint32_t     cleardownTimer_timeout_ms;
    uint8_t      t5_timeout_ms_valid;
    uint16_t     t5_timeout_ms;
    uint8_t      t6_timeout_ms_valid;
    uint16_t     t6_timeout_ms;
    uint8_t      t7_timeout_ms_valid;
    uint16_t     t7_timeout_ms;
    uint8_t      autoAnswer_timeout_ms_valid;
    uint32_t     autoAnswer_timeout_ms;
    uint8_t      deregister_network_timeout_ms_valid;//T10
    uint32_t     deregister_network_timeout_ms;
```

```
uint8_t    dialDurationTimer_timeout_ms_valid;
uint32_t   dialDurationTimer_timeout_ms;
uint8_t    maxDropDialAttempts_valid;
uint32_t   maxDropDialAttempts;
uint8_t    maxDialAttempts_valid;
uint32_t   maxDialAttempts;
uint8_t    intervalBetweenAttempts_valid;
uint16_t   intervalBetweenAttempts;
uint8_t    resetEcallSessionMode_valid;
uint8_t    resetEcallSessionMode;
}ql_ecall_config_t;
```

● 参数

类型	参数	描述
uint8_t	<i>ecallonly_valid</i>	设置 eCall only 模式使能状态 1 使能 0 关闭
uint8_t	<i>ecallonly</i>	eCall only 模式
uint8_t	<i>t5_timeout_ms_valid</i>	设置 T5 定时器使能状态 1 使能 0 关闭
uint16_t	<i>t5_timeout_ms</i>	T5 定时器时间
uint8_t	<i>t6_timeout_ms_valid</i>	设置 T6 定时器使能状态 1 使能设置 0 关闭设置
uint16_t	<i>t6_timeout_ms</i>	T6 定时器时间
uint8_t	<i>t7_timeout_ms_valid</i>	设置 T7 定时器使能状态 1 使能设置 0 关闭设置
uint16_t	<i>t7_timeout_ms</i>	T7 定时器时间
uint8_t	<i>antoAnswer_timeout_ms_valid</i>	设置自动接听超时时间使能状态 1 使能设置 0 关闭设置
uint32_t	<i>antoAnswer_timeout_ms</i>	自动接听超时时间
uint8_t	<i>deregister_network_timeout_ms_valid</i>	设置 T10 定时器使能状态 1 使能设置 0 关闭设置
uint32_t	<i>deregister_network_timeout_ms</i>	T10 定时器时间
uint8_t	<i>dialDurationTimer_timeout_ms_valid</i>	设置重拨总时间定时器使能状态 1 使能设置 0 关闭设置
uint32_t	<i>dialDurationTimer_timeout_ms</i>	重拨总时间

uint8_t	<i>maxDropDialAttempts_valid</i>	设置通话异常挂断重拨次数使能状态 1 使能设置 0 关闭设置
uint32_t	<i>maxDropDialAttempts</i>	通话异常挂断重拨次数
uint8_t	<i>maxDialAttempts_valid</i>	设置最大拨号失败重拨次数使能状态 1 使能设置 0 关闭设置
uint32_t	<i>maxDialAttempts</i>	最大拨号失败重拨次数
uint8_t	<i>intervalBetweenAttempts_valid</i>	设置拨号失败重拨间隔时间使能状态 1 使能设置 0 关闭设置
uint16_t	<i>intervalBetweenAttempts</i>	拨号失败重拨间隔时间
uint8_t	<i>resetEcallSessionMode_valid</i>	设置自动恢复功能使能状态 1 使能设置 0 关闭设置
uint8_t	<i>resetEcallSessionMode</i>	自动恢复功能

3.3.12. ql_ecall_get_config_info

该函数用于获取 eCall 配置。

- 函数原型

```
void ql_ecall_get_config_info(ql_ecall_config_t* ecall_context_info);
```

- 参数

ecall_context_info:

[Out] eCall 配置信息；详见第 3.3.11.1 章。

- 返回值

无

3.3.13. ql_ecall_set_test_number

该函数用于设置 eCall 测试号码。

- 函数原型

```
int ql_ecall_set_test_number(int sim_id, char* number);
```

- 参数

sim_id:

[In] (U)SIM 卡 ID。

- 0 无效卡槽
- 1 (U)SIM1
- 2 (U)SIM2

number:

[In] 测试号码。

- 返回值

QL_ERR_OK 函数执行成功

其他值 函数执行失败，错误码详见 *ql_type.h*。

3.3.14. ql_ecall_set_system_std

该函数用于设置 eCall 系统标准类型。

- 函数原型

```
int ql_ecall_set_system_std(ECALL_SYSTEM_STD_E system_std);
```

- 参数

system_std:

[In] eCall 系统标准类型；详见第 3.3.4.3 章。

- 返回值

QL_ERR_OK 函数执行成功

其他值 函数执行失败，错误码详见 *ql_type.h*。

3.3.15. ql_ecall_get_system_std

该函数用于获取 eCall 系统标准类型。

- 函数原型

```
int ql_ecall_get_system_std(ECALL_SYSTEM_STD_E* system_std);
```

- 参数

system_std:

[Out] eCall 系统标准类型；详见第 3.3.4.3 章。

- 返回值

QL_ERR_OK 函数执行成功

其他值 函数执行失败，错误码详见 *ql_type.h*。

3.3.16. ql_ecall_set_msd_vin

该函数用于设置车辆识别码。

- 函数原型

```
QL_ERR_MSD_RESULT_E ql_ecall_set_msd_vin(msd_Vin_t vin);
```

- 参数

vin:

[In] 车辆识别码；详见第 3.3.16.2 章。

- 返回值

QL_ERR_MSD_SUCCESS 函数执行成功

其他值 函数执行失败，错误码详见第 3.3.16.1 章。

3.3.16.1. QL_ERR_MSD_RESULT_E

MSD 发送结果枚举定义如下：

```
typedef enum
{
    QL_ERR_MSD_NONE = 0,
    QL_ERR_MSD_SUCCESS = QL_ERR_MSD_NONE,
    QL_ERR_MSD_GENERIC = -(QL_ERR_MSD_NONE+1),
    QL_ERR_MSD_BADPARAM = -(QL_ERR_MSD_NONE+2),
    QL_ERR_MSD_VERSION = -(QL_ERR_MSD_NONE+3),
    QL_ERR_MSD_ISOEMI = -(QL_ERR_MSD_NONE+2),
    QL_ERR_MSD_ISOVIDS = -(QL_ERR_MSD_NONE+4),
    QL_ERR_MSD_ISOVIS_MODELYEAR = -(QL_ERR_MSD_NONE+5),
    QL_ERR_MSD_ISOVIS_SEQPLAN = -(QL_ERR_MSD_NONE+6),
    QL_ERR_MSD_LATITUDE = -(QL_ERR_MSD_NONE+7),
    QL_ERR_MSD_LONGITUDE = -(QL_ERR_MSD_NONE+8),
    QL_ERR_MSD_VEH_DIRECTION = -(QL_ERR_MSD_NONE+9),
    QL_ERR_MSD_VEH_LAT_LOCATION1 = -(QL_ERR_MSD_NONE+10),
    QL_ERR_MSD_VEH_LON_LOCATION1 = -(QL_ERR_MSD_NONE+11),
}
```

```

QL_ERR_MSD_VEH_LAT_LOCATION2      = -(QL_ERR_MSD_NONE+12),
QL_ERR_MSD_VEH_LON_LOCATION2      = -(QL_ERR_MSD_NONE+13),
QL_ERR_MSD_VEH_IDENTIFI_NUMBER    = -(QL_ERR_MSD_NONE+14),
QL_ERR_MSD_OPTINAL_DATA_GLONASS    = -(QL_ERR_MSD_NONE+15),
} QL_ERR_MSD_RESULT_E;

```

● 成员

成员	描述
<code>QL_ERR_MSD_SUCCESS</code>	操作成功
<code>QL_ERR_MSD_GENERIC</code>	MSD 类错误
<code>QL_ERR_MSD_BADPARAM</code>	MSD 参数错误
<code>QL_ERR_MSD_VERSION</code>	MSD 版本错误
<code>QL_ERR_MSD_ISOWMI</code>	WMI 识别码错误
<code>QL_ERR_MSD_ISOVDs</code>	VDS 错误
<code>QL_ERR_MSD_ISOVIS_MODELYEAR</code>	车型年份错误
<code>QL_ERR_MSD_ISOVIS_SEQPLAN</code>	装配厂和出厂顺序号错误
<code>QL_ERR_MSD_LATITUDE</code>	纬度错误
<code>QL_ERR_MSD_LONGITUDE</code>	经度错误
<code>QL_ERR_MSD_VEH_DIRECTION</code>	车辆方位错误
<code>QL_ERR_MSD_VEH_LAT_LOCATION1</code>	经度增量 N1 位置错误
<code>QL_ERR_MSD_VEH_LON_LOCATION1</code>	纬度增量 N1 位置错误
<code>QL_ERR_MSD_VEH_LAT_LOCATION2</code>	经度增量 N2 位置错误
<code>QL_ERR_MSD_VEH_LON_LOCATION2</code>	纬度增量 N2 位置错误
<code>QL_ERR_MSD_VEH_IDENTIFI_NUMBER</code>	车辆识别码错误
<code>QL_ERR_MSD_OPTINAL_DATA_GLONASS</code>	GLONASS 可选数据错误

3.3.16.2. msd_Vin_t

车辆识别码结构体定义如下：

```
typedef struct __attribute__((__packed__)) {
    char    isowmi[4];
    char    isovds[7];
    char    isovisModelyear[2];
    char    isovisSeqPlant[8];
} msd_Vin_t;
```

● 参数

类型	参数	描述
char	<i>isowmi</i>	世界制造商识别代码
char	<i>isovds</i>	车辆说明部分
char	<i>isovisModelyear</i>	车型年份
char	<i>isovisSeqPlant</i>	装配厂和出厂顺序号

3.3.17. ql_ecall_get_msd_vin

该函数用于获取车辆识别码。

● 函数原型

```
QL_ERR_MSD_RESULT_E ql_ecall_get_msd_vin(msd_Vin_t* vin);
```

● 参数

vin:

[Out] 车辆识别码；详见第3.3.16.2章。

● 返回值

QL_ERR_MSD_SUCCESS 函数执行成功
其他值 函数执行失败，错误码详见第3.3.16.1章。

3.3.18. ql_ecall_set_msd_version

该函数用于设置 MSD 版本号。

- 函数原型

```
QL_ERR_MSD_RESULT_E ql_ecall_set_msd_version(uint8_t msd_version);
```

- 参数

msd_version:

[In] MSD 版本号。

- 返回值

QL_ERR_MSD_SUCCESS 函数执行成功

其他值 函数执行失败，错误码详见第 3.3.16.1 章。

3.3.19. ql_ecall_get_msd_version

该函数用于获取 MSD 版本号。

- 函数原型

```
QL_ERR_MSD_RESULT_E ql_ecall_get_msd_version(uint8_t* msdVersionPtr);
```

- 参数

msdVersionPtr:

[Out] MSD 版本号。

- 返回值

QL_ERR_MSD_SUCCESS 函数执行成功

其他值 函数执行失败，错误码详见第 3.3.16.1 章。

3.3.20. ql_ecall_set_msd_tx_mode

该函数用于设置 MSD 发送模式（仅适用于 GLONASS 标准）。

- 函数原型

```
QL_ERR_MSD_RESULT_E      ql_ecall_set_msd_tx_mode(QL_ECALL_MSD_TX_MODE_E tx_mode);
```

- 参数

tx_mode:

[In] MSD 发送模式；详见第 3.3.20.1 章。

- 返回值

QL_ERR_MSD_SUCCESS 函数执行成功

其他值 函数执行失败，错误码详见第 3.3.16.1 章。

3.3.20.1. QL_ECALL_MSD_TX_MODE_E

MSD 发送模式枚举定义如下：

```
typedef enum
{
    QL_ECALL_TX_MODE_PULL    = 0,
    QL_ECALL_TX_MODE_PUSH    = 1
} QL_ECALL_MSD_TX_MODE_E;
```

- 成员

成员	描述
<i>QL_ECALL_TX_MODE_PULL</i>	PULL 模式
<i>QL_ECALL_TX_MODE_PUSH</i>	PUSH 模式

3.3.21. ql_ecall_get_msd_tx_mode

该函数用于获取 MSD 发送模式。

- 函数原型

```
void ql_ecall_get_msd_tx_mode(QL_ECALL_MSD_TX_MODE_E* tx_mode);
```

- 参数

tx_mode:

[Out] MSD 发送模式；详见第 3.3.20.1 章。

- 返回值

无

3.3.22. ql_ecall_set_msd_position

该函数用于设置车辆位置信息。

● 函数原型

```
QL_ERR_MSD_RESULT_E ql_ecall_set_msd_position(bool isTrusted, int32_t latitude, int32_t longitude, int32_t direction);
```

● 参数

isTrusted:

[In] 位置是否可信。

1 可信

0 不可信

latitude:

[In] 纬度。

longitude:

[In] 经度。

direction:

[In] 车辆行驶方向。取值范围：0~179，单位：2 度。

● 返回值

QL_ERR_MSD_SUCCESS 函数执行成功

其他值

函数执行失败，错误码详见第 3.3.16.1 章。

3.3.23. ql_ecall_set_msd_position_n1

该函数用于设置车辆经纬度的增量 N1。

● 函数原型

```
QL_ERR_MSD_RESULT_E ql_ecall_set_msd_position_n1(int32_t latitudeDeltaN1, int32_t longitudeDeltaN1);
```

● 参数

latitudeDeltaN1:

[In] 纬度增量 N1。取值范围：-512~511。

longitudeDeltaN1:

[In] 经度增量 N1。取值范围：-512~511。

- 返回值

`QL_ERR_MSD_SUCCESS` 函数执行成功
其他值 函数执行失败，错误码详见第 3.3.16.1 章。

3.3.24. ql_ecall_set_msd_position_n2

该函数用于设置车辆经纬度的增量 N2。

- 函数原型

```
QL_ERR_MSD_RESULT_E ql_ecall_set_msd_position_n2(int32_t latitudeDeltaN2, int32_t longitudeDeltaN2);
```

- 参数

latitudeDeltaN2:

[In] 纬度增量 N2。取值范围：-512~511。

longitudeDeltaN2:

[In] 经度增量 N2。取值范围：-512~511。

- 返回值

`QL_ERR_MSD_SUCCESS` 函数执行成功
其他值 函数执行失败，错误码详见第 3.3.16.1 章。

3.3.25. ql_ecall_set_msd_vehicle_type

该函数用于设置车辆类型。

- 函数原型

```
QL_ERR_MSD_RESULT_E ql_ecall_set_msd_vehicle_type(uint8_t vehicleType);
```

- 参数

vehicleType:

[In] 车辆类型。

- 1 M1 级轿车
- 2 M2 级客车与旅游客车
- 3 M3 级客车与旅游客车
- 4 N1 级轻型商用车
- 5 N2 级重型汽车
- 6 N3 级重型汽车

- 7 L1e 摩托车
- 8 L2e 摩托车
- 9 L3e 摩托车
- 10 L4e 摩托车
- 11 L5e 摩托车
- 12 L6e 摩托车
- 13 L7e 摩托车
- 14 O 拖车
- 15 R 农用车辆
- 16 S 农用车辆
- 17 T 农用车辆
- 18 G 越野车
- 19 SA 房车
- 20 SB 装甲车
- 21 SC 救护车
- 22 SD 灵车
- 23 其他

● 返回值

QL_ERR_MSD_SUCCESS 函数执行成功
其他值 函数执行失败，错误码详见第 3.3.16.1 章。

3.3.26. ql_ecall_get_msd_vehicle_type

该函数用于获取车辆类型。

● 函数原型

```
QL_ERR_MSD_RESULT_E ql_ecall_get_msd_vehicle_type(uint8_t* vehicleType);
```

● 参数

vehicleType:

[Out] 车辆类型。

- 1 M1 级轿车
- 2 M2 级客车与旅游客车
- 3 M3 级客车与旅游客车
- 4 N1 级轻型商用车
- 5 N2 级重型汽车
- 6 N3 级重型汽车
- 7 L1e 摩托车
- 8 L2e 摩托车
- 9 L3e 摩托车

- 10 L4e 摩托车
- 11 L5e 摩托车
- 12 L6e 摩托车
- 13 L7e 摩托车
- 14 O 拖车
- 15 R 农用车辆
- 16 S 农用车辆
- 17 T 农用车辆
- 18 G 越野车
- 19 SA 房车
- 20 SB 装甲车
- 21 SC 救护车
- 22 SD 灵车
- 23 其他

- 返回值

QL_ERR_MSD_SUCCESS 函数执行成功
其他值 函数执行失败，错误码详见第3.3.16.1章。

3.3.27. ql_ecall_set_msd_passengers_count

该函数用于设置乘客数量。

- 函数原型

```
void ql_ecall_set_msd_passengers_count(uint8_t numberOfPassengers);
```

- 参数

numberOfPassengers:
[In] 乘客数量。

- 返回值

QL_ERR_MSD_SUCCESS 函数执行成功
其他值 函数执行失败，错误码详见第3.3.16.1章。

3.3.28. ql_ecall_set_msd_propulsion_type

该函数用于设置车辆推动类型。

- 函数原型

```
void ql_ecall_set_msd_propulsion_type(uint8_t PropulsionType);
```

- 参数

PropulsionType:

[In] 车辆推动类型。

0x1	汽油
0x2	柴油
0x4	天然气
0x8	丙烷气
0x10	电力
0x20	氢气
0x40	其他

- 返回值

QL_ERR_MSD_SUCCESS 函数执行成功
其他值 函数执行失败，错误码详见第 3.3.16.1 章。

3.3.29. ql_ecall_get_msd_propulsion_type

该函数用于获取车辆推动类型。

- 函数原型

```
void ql_ecall_get_msd_propulsion_type(uint8_t* PropulsionType);
```

- 参数

PropulsionType:

[Out] 车辆推动类型。

0x1	汽油
0x2	柴油
0x4	天然气
0x8	丙烷气
0x10	电力
0x20	氢气
0x40	其他

- 返回值

QL_ERR_MSD_SUCCESS 函数执行成功
其他值 函数执行失败，错误码详见第 3.3.16.1 章。

3.3.30. ql_ecall_set_msd_call_type

该函数用于设置 eCall 类型。

- 函数原型

```
void ql_ecall_set_msd_call_type(bool testCall);
```

- 参数

testCall:

[In] eCall 类型。

- 1 测试模式
- 0 正常模式

- 返回值

QL_ERR_MSD_SUCCESS 函数执行成功
其他值 函数执行失败，错误码详见第 3.3.16.1 章。

3.3.31. ql_ecall_get_msd_call_type

该函数用于获取 eCall 类型。

- 函数原型

```
void ql_ecall_get_msd_call_type(bool* testCall);
```

- 参数

testCall:

[Out] eCall 类型。

- 1 测试模式
- 0 正常模式

- 返回值

QL_ERR_MSD_SUCCESS 函数执行成功
其他值 函数执行失败，错误码详见第 3.3.16.1 章。

3.4. 函数使用

3.4.1. 推荐的函数使用流程

eCall 相关 API 函数的使用流程推荐如下三种方式：

方式一：使用移远通信 MSD 编码接口

步骤 1：调用 `ql_ecall_init()` 初始化 eCall；

步骤 2：调用 `ql_ecall_set_user_ind_cb()` 注册上报 eCall 状态的回调函数，以便接收 IVS 上报的 eCall 状态信息；

步骤 3：调用以下接口设置 MSD 参数

1. `ql_ecall_set_msd_version()`
2. `ql_ecall_set_msd_call_type()`
3. `ql_ecall_set_msd_vehicle_type()`
4. `ql_ecall_set_msd_vin()`
5. `ql_ecall_set_msd_propulsion_type()`
6. `ql_ecall_set_msd_passengers_count()`
7. `ql_ecall_set_msd_position()`
8. `ql_ecall_set_msd_position_n1()`
9. `ql_ecall_set_msd_position_n2()`

步骤 4：使用 `ql_ecall_update_msd()` 接口生成 MSD 后，可以调用 `ql_ecall_start_test()`、`ql_ecall_start_manual()`、`ql_ecall_start_automatic()` 接口拨打不同类型的 eCall，测试类型需要拨打前使用 `ql_ecall_set_test_number()` 接口设置测试号码；

步骤 5：在特定场景下（当 eCall 为手动触发，且没有与 PSAP 连接成功且不在发送 MSD 且不在语音通话时），IVS 可以调用 `ql_ecall_hangup()` 挂断 eCall 通话；一般情况为 PSAP 端挂断 eCall。

方式二：用户自行完成 MSD 编码并调用 `ql_ecall_start_test()`、`ql_ecall_start_manual()`、`ql_ecall_start_automatic()` 进行拨号

步骤 1：调用 `ql_ecall_init()` 初始化 eCall；

步骤 2：调用 `ql_ecall_set_user_ind_cb()` 注册上报 eCall 状态的回调函数，以便接收 IVS 上报的 eCall 状态信息；

步骤 3：调用 `ql_ecall_update_msd_raw()` 完成 MSD 的更新；

步骤 4：MSD 更新完成后，可以调用 `ql_ecall_start_test()`、`ql_ecall_start_manual()`、`ql_ecall_start_automatic()` 接口拨打不同类型的 eCall，测试类型需要拨打前使用 `ql_ecall_set_test_number()` 接口设置测试号码；

步骤 5：在特定场景下（当 eCall 为手动触发，且没有与 PSAP 连接成功且不在发送 MSD 且不在语音通话时），IVS 可以调用 `ql_ecall_hangup()` 挂断 eCall 通话；一般情况为 PSAP 端挂断 eCall。

方式三：用户自行完成 MSD 编码并调用 `ql_ecall_dial()` 更新 MSD 和拨号

步骤 1：调用 `ql_ecall_init()` 初始化 eCall；

步骤 2：调用 `ql_ecall_set_user_ind_cb()` 注册上报 eCall 状态的回调函数，以便接收 IVS 上报的 eCall 状态信息；

步骤 3：调用 `ql_ecall_dial()` 在 `p_info` 参数中完成更新 MSD、拨号类型设置；示例：

```
ql_ecall_dial(sim_id, p_info, &id);
```

步骤 4: 在后续 PSAP 需要请求更新时，使用 `ql_ecall_update_msd_raw()`更新 MSD；

步骤 5: 在特定场景下（当 eCall 为手动触发，且没有与 PSAP 连接成功且不在发送 MSD 且不在语音通话时），IVS 可以调用 `ql_ecall_hangup()`挂断 eCall 通话；一般为 PSAP 端挂断 eCall。

3.4.2. 函数使用示例

3.4.2.1. 初始化 eCall

```
void item_ql_ecall_init(void)
{
    int ret = 0;

    printf("test ql_ecall_init: ");
    ret = ql_ecall_init();
    if(ret == QL_ERR_OK)
    {
        printf("ok\n");
    }
    else
    {
        printf("failed, ret = %d\n", ret);
    }
}
```

3.4.2.2. 设置回调函数上报 eCall 状态

```
static void ecall_user_ind_callback(int ind, void *userdata)
{
    printf("\n***** eCall indication Received *****\n");
    printf("ecall_indication: %d - ", ind);

    switch(ind){
        case QL_ECALL_EVENT_SENDING_START:
            printf("QL_ECALL_EVENT_SENDING_START\n");
            break;
        case QL_ECALL_EVENT_SENDING_MSD:
            printf("QL_ECALL_EVENT_SENDING_MSD\n");
            break;
        case QL_ECALL_EVENT_LLACK_RECEIVED:
            printf("QL_ECALL_EVENT_LLACK_RECEIVED\n");
            break;
        case QL_ECALL_EVENT_ALLACK_POSITIVE_RECEIVED:
            printf("QL_ECALL_EVENT_ALLACK_POSITIVE_RECEIVED\n");
            break;
        case QL_ECALL_EVENT_ALLACK_CLEARDOWN_RECEIVED:
            printf("QL_ECALL_EVENT_ALLACK_CLEARDOWN_RECEIVED\n");
            break;
        case QL_ECALL_EVENT_MSUPDATE_PSAP_REQUIRE_UPDATE_MSD :
            printf("QL_ECALL_EVENT_MSUPDATE_PSAP_REQUIRE_UPDATE_MSD\n");
    }
```

```

        break;
    case QL_ECALL_EVENT_MSDUPDATE_PSAP_REQUIRE_TIMEOUT :
        printf("QL_ECALL_EVENT_MSDUPDATE_PSAP_REQUIRE_TIMEOUT\n");
        break;
    case QL_ECALL_EVENT_ACTIVE:
        printf("QL_ECALL_EVENT_ACTIVE\n");
        break;
    case QL_ECALL_EVENT_DISCONNECTED:
        printf("QL_ECALL_EVENT_DISCONNECTED\n");
        break;
    case QL_ECALL_EVENT_ABNORMAL_HANGUP:
        printf("QL_ECALL_EVENT_ABNORMAL_HANGUP\n");
        break;
    case QL_ECALL_EVENT_ONLY_DEREGISTRATION:
        printf("QL_ECALL_EVENT_ONLY_DEREGISTRATION\n");
        break;
    case QL_ECALL_EVENT_MAY_DEREGISTRATION:
        printf("QL_ECALL_EVENT_MAY_DEREGISTRATION\n");
        break;
    case QL_ECALL_EVENT_PSAP_CALLBACK_START:
        printf("QL_ECALL_EVENT_MAY_DEREGISTRATION\n");
        break;
    case QL_ECALL_EVENT_T2_TIMEOUT:
        printf("QL_ECALL_EVENT_T2_TIMEOUT\n");
        break;
    case QL_ECALL_EVENT_T5_TIMEOUT:
        printf("QL_ECALL_EVENT_T5_TIMEOUT\n");
        break;
    case QL_ECALL_EVENT_T6_TIMEOUT:
        printf("QL_ECALL_EVENT_T6_TIMEOUT\n");
        break;
    case QL_ECALL_EVENT_T7_TIMEOUT:
        printf("QL_ECALL_EVENT_T7_TIMEOUT\n");
        break;
    case QL_ECALL_EVENT_ECALL_STARTED:
        printf("QL_ECALL_EVENT_ECALL_STARTED\n");
        break;
    case QL_ECALL_EVENT_INCOMING_CALL:
        printf("QL_ECALL_EVENT_INCOMING_CALL\n");
        break;
    case QL_ECALL_EVENT_DIAL_DURATION_TIMEOUT:
        printf("QL_ECALL_EVENT_DIAL_DURATION_TIMEOUT\n");
        break;
    case QL_ECALL_EVENT_ORI_REDIAL_TIME:
        p_ind_data = (ql_ecall_ind_event_data *)ind_data;
        printf("QL_ECALL_EVENT_ORI_REDIAL_TIME : %d\n",p_ind_data->ori_redial_time);
        break;
    case QL_ECALL_EVENT_DROP_REDIAL_TIME:
        p_ind_data = (ql_ecall_ind_event_data *)ind_data;
        printf("QL_ECALL_EVENT_DROP_REDIAL_TIME : %d\n",p_ind_data->drop_redial_time);
        break;
    case QL_ECALL_EVENT_INTERVAL_TIMEOUT:
        printf("QL_ECALL_EVENT_INTERVAL_TIMEOUT\n");
        break;
    case QL_ECALL_EVENT_AUTO_ANSWER_TIMEOUT:
        printf("QL_ECALL_EVENT_AUTO_ANSWER_TIMEOUT\n");
        break;

```

```

        default:
            printf("UNKNOWN\n");
            break;
    }
}
void item_ql_ecall_set_user_ind_cb(void)
{
    ql_ecall_set_user_ind_cb(ecall_user_ind_callback, NULL);
}

```

3.4.2.3. 拨打 eCall

```

void fast_ecall_dial(void){
    int ret = 0, v = 0;
    ql_voice_ecall_info_t *p_info = NULL;
    char *find = NULL;
    uint32_t id;
    int sim_id;

    p_info = (ql_voice_ecall_info_t *)calloc(1, sizeof(*p_info));
    if (NULL == p_info)
    {
        printf("run out of memory\n");
        return;
    }

    printf("please enter the sim_id: ");
    scanf("%d", &sim_id);
    getchar();

    if(!QL_IS_SIM_VALID(sim_id))
    {
        printf("invalid sim_id\n");
        free(p_info);
        return;
    }

    printf("sim_id is %d\n", sim_id);

    printf("example MSD: 01 04 a9 81 d5 49 70 d6 5c 35 97 ca 04 20 c4 14 60 "
        "0b be 5f 7e b1 4b a6 ee 10 4f c5 27 03 c1 80 q\n");
    printf("please enter MSD(at most 140 hex), end with 'q': ");
    while (1 == scanf("%x", &v))
    {
        p_info->msd[p_info->msd_len++] = v;
    }
    getchar(); // read `q'
    getchar(); // read '\n'

    printf("MSD =====[");
    for(v = 0; v < p_info->msd_len; v++)
    {
        printf("%02x ", p_info->msd[v]);
    }
}

```

```

printf("]\n");

if (p_info->msd_len > QL_VOICE_MAX_ECALL_MSD)
{
    printf("MSD too long\n");
    free(p_info);
    return;
}

printf("please enter eCall type(1 - test, 2 - emergency, 3 - reconfig): ");
scanf("%d", (int *)&p_info->type);
getchar();

printf("please enter test number(emergency ecall should be empty): ");
find = NULL;
fgets(p_info->test_number, sizeof(p_info->test_number)-1, stdin);
find = strchr(p_info->test_number, '\n');
if(find)
{
    *find = '\0';
}

printf("how to trigger eCall(0 - manual, 1 - auto): ");
scanf("%d", &p_info->auto_trigger);
getchar();

ret = ql_ecall_dial(sim_id, p_info, &id);
if (ret == QL_ERR_OK)
{
    printf("ok, call_id is %u\n", id);
}
else
{
    printf("failed, ret = %d\n", ret);
}

free(p_info);
}

void item_ql_ecall_dial(void)
{
    printf("test ql_voice_ecall_dial: \n");
    printf("Dialling fast ecall\n");
    fast_ecall_dial();
}

```

3.4.2.4. 使用原始数据更新 MSD

```

void item_ql_ecall_update_msd_raw(void)
{
    int ret = 0, v = 0;
    uint32_t msd_len = 0;
    char msd[QL_VOICE_MAX_ECALL_MSD] = {0};
}

```

```

printf("test ql_voice_ecall_update_msd: \n");
printf("example MSD: 01 04 a9 81 d5 49 70 d6 5c 35 97 ca 04 20 c4 14 60 "
"0b be 5f 7e b1 4b a6 ee 10 4f c5 27 03 c1 80 q\n");
printf("please enter MSD(at most 140 hex), end with 'q': ");
while (1 == scanf("%x", &v))
{
    if(msd_len >= QL_VOICE_MAX_ECALL_MSD)
    {
        printf("MSD too long\n");
        int c;
        while ((c = getchar()) != '\n' && c != EOF) { }
        return;
    }
    else
    {
        msd[msd_len++] = v;
    }
}
getchar(); // read `q'
getchar(); // read '\n'

ret = ql_ecall_update_msd_raw((uint8_t *)msd, msd_len);
if (ret == QL_ERR_OK)
{
    printf("ok\n");
}
else
{
    printf("failed, ret = %d\n", ret);
}
}

```

3.4.2.5. 更新 MSD

```

void item_ql_ecall_update_msd (void)
{
    int ret = 0;
    ret = ql_ecall_update_msd();
    if (ret == QL_ERR_OK)
    {
        printf("ok\n");
    }
    else
    {
        printf("failed, ret = %d\n", ret);
    }
}

```

3.4.2.6. 挂断 eCall

```
void item_ql_ecall_hangup(void)
{
    int ret = 0;
    int sim_id;

    printf("test ql_voice_ecall_hangup: \n");
    ret = ql_ecall_hangup();
    if (ret == QL_ERR_OK)
    {
        printf("ok\n");
    }
    else
    {
        printf("failed, ret = %d\n", ret);
    }
}
```

3.4.2.7. 设置车辆识别码

```
void item_ql_ecall_set_msds_vin(void)
{
    int ret = 0;
    uint8_t index = 0;
    msd_Vin_t vin;
    char* find = NULL;
    char vin_str[QL_ECALL_MAX_VIN];

    printf("Input: default(0) other(1)\n");
    scanf("%hhd", &index);
    getchar();
    if(index)
    {
        printf("Please insert VIN:\n");
        fgets(vin_str, QL_ECALL_MAX_VIN-1, stdin);
        find = strchr(vin_str, '\n');
        if(find)
        {
            *find = '\0';
        }
        // example 1: WM9VDSVDSYA123456
        // example 2: 4Y1SL65848Z411439
        // example 3: VF37BRFVE12345678
        memcpy(&vin.isowmi, vin_str, 3);
        memcpy(&vin.isovds, (vin_str+3), 6);
        memcpy(&vin.isovisModelyear, (vin_str+9), 1);
        memcpy(&vin.isovisSeqPlant, (vin_str+10), 7);
    }
    else
    {
        memcpy(&vin.isowmi, "WM9", 3);
        memcpy(&vin.isovds, "VDSVDS", 6);
    }
}
```

```

        memcpy(&vin.isovisModelyear,"Y",1);
        memcpy(&vin.isovisSeqPlant,"A123456",7);
    }

    ret = ql_ecall_set_msd_vin(vin);
    printf(" ql_ecall_set_msd_vin ret = %d\n", ret);
}

```

3.4.2.8. 发起 eCall 测试

```

void item_ql_ecall_start_test(void)
{
    int ret = 0;
    int sim_id;

    printf("please enter the sim_id: ");
    ret = scanf("%d", &sim_id);
    getchar();

    if(!QL_IS_SIM_VALID(sim_id))
    {
        printf("invalid sim_id\n");
    }

    ret = ql_ecall_start_test(sim_id);
    printf(" ql_ecall_start_test ret = %d\n", ret);
}

```

3.4.2.9. 发起手动拨打 eCall

```

void item_ql_ecall_start_manual(void)
{
    int ret = 0;
    int sim_id;

    printf("please enter the sim_id: ");
    ret = scanf("%d", &sim_id);
    getchar();

    if(!QL_IS_SIM_VALID(sim_id))
    {
        printf("invalid sim_id\n");
    }

    ret = ql_ecall_start_manual(sim_id);
    printf(" ql_ecall_start_manual ret = %d\n", ret);
}

```

3.4.2.10. 发起自动拨打 eCall

```
void item_ql_ecall_start_automatic(void)
{
    int ret = 0;
    int sim_id;

    printf("please enter the sim_id: ");
    ret = scanf("%d", &sim_id);
    getchar();

    if(!QL_IS_SIM_VALID(sim_id))
    {
        printf("invalid sim_id\n");
    }

    ret = ql_ecall_start_automatic(sim_id);
    printf(" ql_ecall_start_automatic ret = %d\n", ret);
}
```

4 附录 参考文档及术语缩写

表 4: 参考文档

文档名称
[1] Quectel_AG35-CET&AG35-EUT_QuecOpen(SDK)_快速开发指导
[2] BS EN 16454-2015

表 5: 术语缩写

缩写	英文全称	中文全称
ACK	Acknowledgment	确认消息
AL-ACK	Application Layer Acknowledgement	应用层确认消息
API	Application Programming Interface	应用程序编程接口
eCall	emergency Call	紧急电话
GLONASS	Global Navigation Satellite System (Russia)	格洛纳斯导航卫星系统（俄罗斯）
GNSS	Global Navigation Satellite System	全球导航卫星系统
GSM	Global System for Mobile Communication	全球移动通信系统
HLACK	High Layer Acknowledgement	高级应用层确认消息
IoV	Internet of Vehicles	车联网
IVS	In-vehicle System	车载系统
LLACK	Link Layer Acknowledgement	链路层确认消息
MO	Mobile Originated	终端发起
MSD	Minimum Set of Data	最小数据集
MT	Mobile Terminated	终端终止

NACK	Negative Acknowledgement	否定消息
PSAP	Public Safety Answering Point	公共安全应答中心
SDK	Software Development Kit	软件开发工具包
UMTS	Universal Mobile Telecommunications System	通用移动通信系统
(U)SIM	(Universal) Subscriber Identity Module	(通用)用户身份识别模块
VDS	Vehicle-Descriptive-Section	车辆说明部分
VIN	Vehicle Identification Number	车辆识别码
WMI	World Manufacturer Identifier	世界制造厂识别代号

Preliminary
Quectel Confidential